Installation manual

What to install on a new computer, in what order, and how to know it worked — before opening the application for the first time. Once installed, how it works screen by screen is in the **User manual** (`docs\MANUAL.html`).

Created by Marcelo Barbosa. © 2026 Marcelo Barbosa. The application and its manuals are free software under the GNU AGPL-3.0 (`LICENSE.txt`): anyone may use, copy, change and share them, with no authorisation needed, as long as the author credit is kept (`NOTICE.txt`).

Ten minutes, no internet, nothing to install first, and no administrator rights needed. Python comes inside the package, along with everything else; the computer only has to be a 64-bit Windows — or an Apple Silicon Mac, with its own package (Installing on a Mac). With administrator rights run `setup.bat` ; without them, `setup-noadmin.bat` — the same installation, for your account only.

What is in the package

One file: `ner-analyzer-4.8.1-windows.zip` , about 100 MB, accompanied by `ner-analyzer-4.8.1-windows.zip.sha256` to check the copy.

INSIDE THE ZIP	WHAT IT IS
`setup.bat`	The installer, and the only file to run: installs into a folder of its own, replaces a previous version keeping its data, registers the tasks, starts the server, opens the browser and then deletes the unzipped package and the zip. Section Installing.
`setup-noadmin.bat`	The same installer without the Windows administrator prompt (`setup.bat --no-admin`): for a managed computer where the rights cannot be granted. It installs for the account signed in, in that account's profile, and nothing else differs.
`install.bat · start.bat · stop.bat`	The pieces `setup.bat` runs underneath. Opening `install.bat` in the unzipped package hands over to `setup.bat` , so it does the same thing.
`autostart.bat`	Start at sign-in and install the watchdog; `autostart.bat remove` undoes it.
`backup.bat · run-tests.bat · build_package.bat`	Backup and restore; the test suite; building a new package from this installation.
`reset-password.bat`	A new password for an account that forgot it, on this computer (Mac: `mac/reset-password.command`). See Common problems.
`backend\ · frontend\ · tests\ · tools\`	The application: server, compiled screens (`backend\app\static`), tests and operating scripts.
`vendor\python\`	Python 3.12 itself (64-bit): the official python.org *embeddable package*, 11 MB, signed by the Python Software Foundation. The installer unpacks it into `.python\` . It is what makes installing Python unnecessary.
`vendor\wheels\`	Every Python package the application depends on, already downloaded, built for that Python 3.12 on 64-bit Windows — and `pip` itself, which the installer uses to install them. It is what makes the internet unnecessary.
`docs\`	This manual, the one-page *Quick install* (`QUICK-INSTALL.html`) and the User manual (HTML and PDF), the project history and the technical notes.
`VERSION · CHANGELOG.md · PACKAGE.txt · MANIFEST.txt`	The version, what changed in each version, the package's record (version, date, counts) and the file list with each file's hash.
`LICENSE.txt · NOTICE.txt · THIRD-PARTY-NOTICES.txt`	The licence (GNU AGPL-3.0, full text); the author credit *Created by Marcelo Barbosa*, kept as an additional term of the licence (section 7(b)); and the licence of every third-party component the package carries.
`data\`	Empty, holding only a `.keep` marker. It is where the database and the uploaded documents will be born. `setup.bat` does not copy it into the target folder.

THE PACKAGE CARRIES NO DATA

No projects, no drawings, no TEAP, no accounts and no password hashes. That is a security property, not tidiness: the builder **refuses** a package that includes the `data` folder, and checks the file list before writing the zip. The practical consequence: the USB stick with the package is just software, and can travel like any installer and be shared freely under the licence (Version and licence).

What the computer needs

ITEM	WHY
`Windows 10 / 11, 64-bit`	The compiled packages in the zip are for 64-bit Windows. (A Mac with Apple Silicon has its own package: section Installing on a Mac.)
`Visual C++ runtime`	The numeric packages use Microsoft's `msvcp140.dll`, which virtually every Windows already has (any *Microsoft Visual C++ Redistributable 2015–2022* installs it, and almost every program brings one). When it is missing, the installer places the copy bundled inside the packages next to Python, with no administrator — and says on screen which of the two happened. There is nothing to install beforehand.
`~1 GB of disk`	About 350 MB of that is the private Python (`.python\`) the installer builds inside the folder; the rest is the package and room for the first books.
`Administrator rights`	**Not required.** Nothing in the installation needs them: the application goes into the person's own profile, its two scheduled tasks are per-user, the server listens on `127.0.0.1` only. `setup.bat` asks Windows for them when it starts, because on some managed computers a script run without them is refused by policy before it starts; when they are refused — or cannot be granted at all — it offers to install **for this account only**, which is exactly what it does with them. `setup-noadmin.bat` (or `setup.bat --no-admin`) skips the question. If they are granted, they must belong to **the person who will use the application**, signed in as themselves: typing another account's administrator name and password into the prompt would install the application in that other account's profile, and setup refuses it (section Common problems).
`A browser`	Google Chrome, Microsoft Edge or Mozilla Firefox, in a current version. Safari and Internet Explorer are not supported.
`Memory`	8 GB works; 16 GB leaves room to export a 75-page book with the browser and Outlook open.

That is the list. **No Python installed**, no internet, no database server, no web server, no Node.js, no Docker — and no artificial intelligence of any kind: nothing in the folder calls out to any model, service or account.

Python comes in the package

There is nothing to install before the application. The Python it uses lives inside its own folder.

The application is a server written in Python. Rather than require every computer to have Python installed — a download, an installer, a PATH option to tick — the package carries in `vendor\python\` the official python.org **embeddable package**: the complete Python 3.12 for 64-bit Windows, in an 11 MB zip made precisely to be shipped inside a program. `install.bat` unpacks it into `.python\`, beside `backend\`, and installs the packages from `vendor\wheels` into it.

CONSEQUENCE	IN PRACTICE
`Nothing outside the folder`	That Python is not added to PATH, does not appear in *Programs and Features*, does not touch the registry. Deleting the folder deletes the Python with it.
`Isolated from the rest of the machine`	The file `.python\python312._pth` is that Python's entire search path: the standard library, the packages installed in `.python\Lib\site-packages` and the application's `backend\`. Another Python installed on the machine, environment variables such as `PYTHONPATH` and user packages are ignored. Two Pythons on the same machine never see each other.
`If a Python is already there`	A Python installed on the machine is not used, not changed and not required: with `vendor\python\` in the package, the installer always uses the embedded Python. A `.python\` already in the folder (a previous installation) is deleted and unpacked again from the package's zip; if the server has it in use, the installer stops with `The previous runtime is in use` — `stop.bat` and run it again. (`setup.bat` never meets this: in *update* mode it stops the server and moves the whole old installation aside before installing.)
`Provenance checked`	The package builder only accepts the python.org zip whose SHA-256 is pinned in `tools\make_package.py`, and the installer checks the digital signature of `python.exe` (*Python Software Foundation*) after unpacking, printing the publisher on screen.
`Fixed version: 3.12.10`	The packages in `vendor\wheels` are built for Python 3.12, and 3.12.10 (April 2025) is the last 3.12 published with Windows binaries — later ones are source-only security releases. Changing the Python means changing the whole package set, and is a new version of the application, not a file swap.

> FOR THE DEVELOPER
>
> A copy of the code without `vendor\python\` (a development checkout) makes `install.bat` use the Python installed on the machine (3.11 or newer, 64-bit, preferably 3.12) and build a virtual environment in `.venv\`. Every script looks for `.python\` first and `.venv\` second, so the two arrangements coexist and the package built on a development machine always carries the embedded Python.

Installing: setup.bat

Unzip the file anywhere, right-click `setup.bat` and choose *Run as administrator* — or, on a computer where you cannot get administrator rights, double-click `setup-noadmin.bat`. When it finishes, the application is installed in a folder of its own

and running, the browser is open on it, and the installation files (the unzipped folder and the zip itself) are gone. Only the application is left.

1. **Unblock the zip** (optional, but it saves a Windows prompt per script): right-click `ner-analyzer-4.8.1-windows.zip` → *Properties* → tick *Unblock* at the bottom → *OK*. Without it, Windows may show *Open File – Security Warning* before the script runs; click *Run*.
2. **Unzip** the file: right-click → *Extract All...*. Where does not matter: the Downloads folder is fine, because nothing will stay there. Do not run anything from inside the zip without extracting it. You get a `ner-analyzer` folder with `setup.bat` inside.
3. Right-click `setup.bat` → **Run as administrator**, and answer *Yes* to the Windows prompt. A plain double-click works too: setup notices it has no administrator rights, asks Windows for them, and carries on in a new window. **Without administrator rights** — a managed computer where they cannot be granted — double-click `setup-noadmin.bat` instead: the same installation, for your account only, with no prompt. The window shows the source folder, the target folder and the mode (*install* or *update*), then runs by itself through six numbered steps. Two to five minutes, no internet.
4. The browser opens at `http://127.0.0.1:8000` as soon as the server answers, and an **NER Analyzer** shortcut appears on the desktop. On a new installation, create the first account (section First sign-in).
5. The last screen reads `Installed - version 4.8.1` and the folder the application went to; on an update, also the version it replaced and where the old one was kept. Press a key: the window closes and, about three seconds later, the unzipped package and the zip are deleted.

STEP	WHAT HAPPENS
before [1/6]	Two checks. **Administrator rights**: without them, `Administrator rights are needed. Windows will ask for them now...`, the Windows prompt, and the setup continues in a new window (`The setup continues in the new window. This one can be closed.`). If the prompt is refused: `[X] The administrator rights were not granted.` and the question `Install without administrator rights, for this account only? [Y/N]` — `Y` carries on in this window, for this account (`Installing without administrator rights, for this account only.`); `N` installs nothing. With `--no-admin`, or from `setup-noadmin.bat`, the prompt is skipped and the same line is shown. **The right person**: if the window runs as another account than the person signed in (someone typed an administrator's name and password into the prompt), setup stops with `[X] This window runs as another account than the person signed in.` and installs nothing.
[1/6]	`New installation.`, or `Replacing version 2.x.y - its data is kept.` when the target folder already holds an installation (recognised by `.python\python.exe`, `.venv\Scripts\python.exe` or `data\ner.sqlite3`). When replacing, in this order: both scheduled tasks are disabled, so that neither starts the old version while it is being replaced; the database is backed up (`Backup of the database first (backup.bat)...`) into `data\backup-YYYYMMDD-HHMMSS\`; the server is stopped; the **old installation is moved aside, whole**, to `ner-analyzer.previous` next to it (a `.previous` left by an earlier update is removed first); and its `data\` folder is moved into the new, empty installation folder. No file of the old version survives into the new one, and nothing of the old version is deleted. If the backup fails, or the old folder cannot be moved because something still has it open, setup stops with an `[X]`, says `Nothing was changed.` and re-enables the tasks.
[2/6]	`Copying the package into the target folder...`: everything but `setup.bat` and `setup-noadmin.bat` themselves and the zip's empty `data\` folder. The copied files lose the Windows "downloaded from the internet" mark, so the installed scripts run without a security prompt.
[3/6]	`Installing (install.bat in the target)...`: `install.bat` runs *in the target folder* and shows its own steps, also numbered `[1/6]` to `[6/6]`: the package's Python unpacked into `.python\`, `pip`, the packages from `vendor\wheels`, and the proof that every component loads (`all components load`). It is section Python comes in the package, with nothing to do by hand.
[4/6]	`Registering the sign-in task and the watchdog (autostart.bat)...`: the sign-in task **NER Spatial Coverage Analyzer** and the watchdog **NER Analyzer watchdog**, both for the person signed in and both set to survive a laptop on battery (section Starting by itself, and the watchdog). With `--no-services` the line reads `No scheduled task and no server: --no-services.` and setup goes straight to `[6/6]`. If `autostart.bat` reports a problem, `[*] autostart.bat reported a problem (above).`: the application stays installed and setup moves on to `[6/6]`.
[5/6]	`Starting the server through the scheduled task...` (not from the setup window, so closing it does not bring the server down), then up to 90 seconds of waiting for `http://127.0.0.1:8000/healthz`. When it answers, the default browser opens on it, as the person signed in and not with the setup window's administrator rights, and the **NER Analyzer** desktop shortcut is written. If the server does not answer in time, `[*] The server did not answer within 90 s.`; the shortcut is written all the same.
[6/6]	`Removing the installation files...`: first `vendor\` inside the installed folder (about 100 MB of packages and the Python zip, which the running application never reads), always; then, once the window closes, every file of the unzipped package (by the package's own `MANIFEST.txt` list, never a whole folder), its `vendor\` folder, the folders left empty, and `ner-analyzer-4.8.1-windows.zip` with its `.sha256`, wherever they are: in the unzipped folder, one or two levels up, or in Downloads.

> **IF A STEP FAILS AFTER THE OLD VERSION WAS MOVED ASIDE**
>
> The copy, `install.bat` or the runtime check failing in *update* mode puts everything back: the `data\` folder returns to the old installation, the half-built new folder is removed, `ner-analyzer.previous` is moved back to `ner-analyzer`, the tasks are re-enabled and the old server is started (`Version 2.x.y is back in …, with its data.`). The data is never deleted at any point: it moves with the installation, and its backup is inside it. If even the move back fails, setup says where each part is and deletes nothing.

HOW TO CALL IT	EFFECT
`setup.bat`	Installs into `%LOCALAPPDATA%\Programs\ner-analyzer`, that is `C:\Users\<you>\AppData\Local\Programs\ner-analyzer`: the folder Windows reserves for per-user programs, never synced by OneDrive.

HOW TO CALL IT	EFFECT
`setup.bat D:\Apps\ner-analyzer`	Installs into the given folder (from a Command Prompt run as administrator, in the unzipped folder). Keep it outside OneDrive, Dropbox and network drives: for OneDrive and Dropbox setup shows a `[*]` warning, but does not refuse. To update an installation made this way, give the same folder again.
`setup.bat --no-services`	Installs and cleans up, but registers no task, starts nothing and writes no desktop shortcut. For whoever will start with `start.bat`, or to test an installation without touching the tasks of another one under the same user (the task names are fixed). Tasks that already existed and were disabled at `[1/6]` are re-enabled as they were.
`setup.bat --keep-installer`	Does everything, but leaves the unzipped package and the zip where they are. The `vendor\` inside the installed folder is removed all the same.
`setup.bat --no-admin setup-noadmin.bat`	Does not ask Windows for administrator rights: installs for this account only, in the person's own profile, with the sign-in task and the watchdog registered for that account. For a managed computer where the rights cannot be granted. Everything else is as with `setup.bat`; `setup-noadmin.bat` is the double-click form of it and passes any other option along.
`setup.bat --help`	Shows the options (also `-h` and `/?`) and installs nothing. The options can be combined, in any order, with the target folder.

WHAT IT REFUSES

Running from a copy that is not a built package (no `MANIFEST.txt`, or no Python under `vendor\python`). Installing into the unzipped folder itself, inside it, or into a folder that contains it (the unzipped folder is deleted at the end). Running as another account than the person signed in. When a step stops with an `[X]` message, **the package is not deleted**: the message says why, the tasks held in *update* mode are re-enabled, and the package can be run again once the cause is fixed (Common problems). The `[*]` warnings of steps `[4/6]` and `[5/6]` do not stop the installation.

WHY THE CLEAN-UP

Deleting the zip and the unzipped folder keeps the machine tidy: one installed copy, and no stale installer left behind to be run again by mistake. It is not there to stop copies: anyone may copy and share the package under the licence (`LICENSE.txt`, `NOTICE.txt` and section Version and licence). To keep the package, run `setup.bat --keep-installer`.

AFTER SETUP.BAT THERE IS NO VENDOR\ IN THE INSTALLED FOLDER

Deleted on purpose: it is installation weight, not application weight. The consequence: the installed folder's `install.bat` **cannot** rebuild `.python\` on its own; do not run it there. To repair or update an installation, run the `setup.bat` of a package (the same version or a newer one). A record of what `setup.bat` did is in `data\setup.log`.

install.bat and start.bat

In the installation package, `install.bat` is not a second way in: opening it hands over to `setup.bat` (`This is the installation package: handing over to setup.bat…`), so whoever opens it by habit gets the same installation, in the same folder, with the same clean-up.

`install.bat` installs in place only where there is no `setup.bat` beside it, or no `MANIFEST.txt`: in a development checkout, where it builds a `.venv\` with the Python installed on the machine (section Python comes in the package, *For the developer*), and when `setup.bat` itself calls it in the target folder. It writes nothing outside its folder, and it can be run again at any time with the server stopped: it rebuilds the private Python from scratch and never touches `data\`.

`start.bat` is not needed after `setup.bat`: the server is already running with no window, and starts by itself at every sign-in. It stays in the installed folder for whoever wants the server in a console window (closing the window stops it), for example to watch it start after a problem.

Installing on a Mac

The same application, packaged for **Apple Silicon** (M1 or later) on **macOS 11 or later**: `ner-analyzer-4.8.1-macos-arm64.zip`, about 110 MB, with `ner-analyzer-4.8.1-macos-arm64.zip.sha256` to check the copy (`shasum -a 256` in Terminal). No administrator password, no internet, nothing to install first.

IN THE PACKAGE	WHAT IT IS
`setup.command`	The installer — the macOS twin of `setup.bat`.
`vendor/python/`	A standalone CPython 3.12.15 for Apple Silicon (python-build-standalone, release 20261009), pinned by its SHA-256 when the package is built.
`vendor/wheels/`	The same 48 Python packages, at the same versions, as the Windows package, built for macOS on Apple Silicon (`requirements-lock-mac.txt`).
`mac/`	`start.command` , `stop.command` , `backup.command` , `run-tests.command` , `autostart.command` , `uninstall.command` — they stay in the installed folder.

Steps

1. Double-click the zip in Downloads (Archive Utility unzips it).
2. Open **Terminal**, type `bash` and a space, drag `setup.command` from the unzipped `ner-analyzer` folder into the window, press `Return` . To choose another folder, add it after the script: `bash …/setup.command ~/Documents/ner-analyzer` .
3. The browser opens at `http://127.0.0.1:8000` ; create the first account (First sign-in).

Double-clicking `setup.command` in Finder works too, but a file downloaded from the internet that Apple has not notarised is held back the first time: macOS says it *could not verify* the developer. Click *Done*, open *System Settings → Privacy & Security*, scroll to the message about `setup.command` and click *Open Anyway*. Running it through `bash` in Terminal, as above, does not ask.

What setup.command does

STEP	WHAT HAPPENS
[1/6]	Checks the Mac (Apple Silicon, macOS 11 or later) and the package, and takes the downloaded-from-the-internet mark off the unzipped files (they are your own files: no password). Default folder `~/Applications/ner-analyzer` . An existing installation there means *update*.
[2/6]	On an update: backs up the database into the installation's `data/backup-…` , stops its server, moves it aside to `ner-analyzer.previous` and carries its `data` folder into the new one. If anything later fails, the previous version is put back, data first.
[3/6]	Copies the application (never a `data` folder, never the installer itself).
[4/6]	Unpacks Python into `.python/` and installs the bundled packages into it, offline (`pip --no-index --no-deps` , the user's own Python packages ignored), then checks every component loads.
[5/6]	Registers a **launch agent** for your account (`~/Library/LaunchAgents/com.marcelobarbosa.ner-analyzer.plist`): the server starts now, at every sign-in, and again if it stops on its own — on a Mac launchd does what the watchdog task does on Windows. Waits for `/healthz` , opens the browser, writes `NER Analyzer.webloc` on the Desktop. With `--no-services` nothing is started.
[6/6]	Removes `vendor/` from the installation and `ner-analyzer.previous` , then the unzipped package and `ner-analyzer-4.8.1-macos-arm64.zip` with its `.sha256` (beside it, one level up, or in Downloads). `--keep-installer` keeps them. The log is `data/setup.log` .

Afterwards

Stop the server with `mac/stop.command` (it starts again at the next sign-in), start it with `mac/start.command` , back up with `mac/backup.command` (the same tool as `backup.bat`), run the tests with `mac/run-tests.command` . `mac/uninstall.command` stops it, removes the launch agent and the Desktop link, and deletes the application — the `data` folder only if you type `delete` . The data, the backups and the restore work exactly as on Windows: see Where the data lives and Backup and restore. A backup made on Windows restores on a Mac and the other way round.

> NOT TESTED ON A MAC BY THE BUILDER
>
> The macOS package was built and its installer rehearsed end to end on Linux (same shell steps, a Linux runtime in place of the Mac one: install, update with backup, a broken package rolled back, tests passing in the installed copy, the server answering). The first install on a real Mac is the remaining check: if a step stops, the window says which and `data/setup.log` has the detail. A company-managed Mac may forbid scripts from Downloads or launch agents; IT has to allow them.

First sign-in

1. At the end of `setup.bat` the browser is already open at `http://127.0.0.1:8000` . Later, open it with the **NER Analyzer** shortcut on the desktop, or type that address into Chrome, Edge or Firefox. Right after a reboot, give the server 20 to 40 seconds to start.
2. The first screen is **Create the first account**: username, display name, password (at least **10 characters**; a phrase you remember beats a scramble of symbols) and the

password again. That account is an administrator and creates the others. **There is no default user and no default password.**

3. You land on the `Projects` tab. The `PT-BR` / `EN` button in the top bar switches the interface language. From here on, the **User manual**.

> ANOTHER PORT
>
> If port 8000 is already taken by another program, `start.bat --port 8080` starts on another one. For the sign-in task and the watchdog, set the environment variable `NER_PORT` to the same port before running `autostart.bat`. The desktop shortcut always points at port 8000.

Starting by itself, and the watchdog

Optional, but this is how the application behaves on a working machine: no window, up after any reboot, and back within two minutes should it ever stop answering.

1. Double-click **`autostart.bat`**. It creates two Windows scheduled tasks for the person signed in (`setup.bat` runs it for you): **NER Spatial Coverage Analyzer** (at that person's sign-in, starts the server with no window; log in `data\server.log`) and **NER Analyzer watchdog** (every two minutes, checks that the server answers and starts it again if not; log in `data\watchdog.log`). Neither needs administrator rights.
2. To start it now without signing out: `schtasks /run /tn "NER Spatial Coverage Analyzer"` in a Command Prompt — or simply wait for the watchdog, which starts it within two minutes.
3. To stop the server: `stop.bat`. To undo both tasks: `autostart.bat remove`.

The task fires *at sign-in*, not at boot: the application reads and writes a folder in your profile, so it runs as you and needs no stored password. It fires at *your* sign-in only — it is registered for the account that ran `autostart.bat` (`tools\signin-task.ps1`), which is what lets an account without administrator rights create it; a task for every user of the computer is an administrator's to create. The watchdog goes through a `.vbs` precisely so that no black window ever flashes on screen every two minutes.

Should Windows refuse the sign-in task all the same, `autostart.bat` says so (`[*] Windows refused the sign-in task`) and installs the watchdog regardless. The application then still comes up by itself: the watchdog starts the server within two minutes of signing in, and keeps it up.

Checking it works

With the server running, open `http://127.0.0.1:8000` and confirm:

- the sign-in screen appears, with the *Where this data lives* statement and *version 4.8.1* under the title;
- you create the administrator account and reach the `Projects` tab;
- the left-hand navigation lists the steps from `Import` to `Reports`, and the sidebar reads `version 4.8.1`;
- `http://127.0.0.1:8000/healthz` answers `{"status":"ok","version":"4.8.1"}`.

For a deeper check, double-click **`run-tests.bat`**: around 900 tests, one to two minutes. All passing (*All tests passed*), the installation is sound — every component loads, the rules compute what the manual says, the exports have the shape the manual describes.

Where the data lives

PATH	WHAT IT IS

`data\ner.sqlite3`
`data\ner.sqlite3-wal`
`data\ner.sqlite3-shm`
	The database: projects, spaces, NERs, matches, analyses, accounts and audit log. The `-wal` and `-shm` files hold recent changes while the server runs — copying just the `.sqlite3` by hand silently loses work; use `backup.bat`.

`data\uploads\project_<id>\`	The PDF books and TEAP workbooks exactly as they were uploaded.
`data\criticality-matrix.xlsx`	The criticality matrix, on an installation where one was loaded: one file for the whole installation, outside any project. Most installations have none.
`data\exports\project_<id>\`	The generated reports (marked-up PDF, workbook, CSV). They can be deleted; they are regenerable.

PATH	WHAT IT IS
`data\server.log · data\watchdog.log · data\watchdog-heartbeat.txt`	The server log, the watchdog log (failures only) and the watchdog heartbeat (one line, rewritten every two minutes).
`data\setup.log`	What `setup.bat` did: mode, source folder, target folder, services, what was deleted. Only on installations it made.
`data\backup-…\`	The backups made by `backup.bat`.
`.python\`	The installation's private Python, with the packages installed. The installer rebuilds it from `vendor\`; on an installation made by `setup.bat`, which keeps no `vendor\`, a package's `setup.bat` rebuilds it. To carry the application to another machine, carry the zip, not this folder. (On a development checkout the folder is `.venv\`.)

Everything stays inside the application folder. The server listens on `127.0.0.1` only, makes no outbound connection, and there is no encryption at rest: the database is protected exactly as far as this machine's disk and Windows account are. The full statement is on the application's sign-in screen.

Backup and restore

COMMAND	WHAT IT DOES
`backup.bat`	Writes `data\backup-YYYYMMDD-HHMMSS\` with a **consistent** copy of the database (through SQLite's backup API — valid even while the server is running), the `uploads` folder and, when the installation has one, `criticality-matrix.xlsx`. A `BACKUP.txt` inside says the version, the time, the size and whether the matrix is in it.
`backup.bat --to D:\NER-backups`	The same, into a folder or drive of your choice — which is what makes the backup survive the disk.
`backup.bat --restore data\backup-…`	Puts a backup back. **With the server stopped** (`stop.bat`) — the script refuses while it is answering. The database and uploads that were there are moved to `data\replaced-…\`, not deleted; delete that folder once you are sure. The installation's own criticality matrix is replaced only when the backup carries one. The next time the server starts, it points the record of every uploaded file at this installation's `data\uploads`, whichever folder the backup was taken in.

When: before deleting a project, before installing a new version, and periodically to an external drive. Backups under `data\` accumulate; delete old ones from time to time.

Carrying projects from another machine

One project, to a colleague's computer — the usual case when a venue changes hands: `Export` on the *Projects* screen writes the project — its books, its TEAP and every decision — into one `.zip`; on the other computer, *Import a project package* on the same screen brings it in as a new project of whoever imports it, and the analysis is run again there. No account travels, nothing on the receiving computer is overwritten, and the two copies are independent from then on. The handbook explains it under *Projects → The project package*. The receiving computer needs the same version of the app or a newer one.

Copying everything — every project and every account — is what you do to change computers without losing anything:

1. On the source machine, `backup.bat --to E:\` (a USB stick or a network folder). The backup holds the database, the uploads and the criticality matrix when there is one.
2. On the new machine, install the application (sections above) and do **not** create the first account yet — or do; it will be replaced by the accounts in the backup.
3. With the server stopped, `backup.bat --restore E:\backup-…`. Start it. The database migration, if the new version needs one, runs by itself on first start — and on that start the server points the record of every drawing and TEAP at this installation's own `data\uploads`, so the folder, the drive and the Windows account need not be the ones of the source machine. Each record is moved only when the file is here and is the same file (same SHA-256); each move is written to the project's audit log.

> TWO THINGS BEFORE COPYING THE DATABASE
>
> First: the copy carries **the accounts with their password hashes**, along with every project, drawing and the audit log. The USB stick becomes the database — treat it as one, and wipe it afterwards.
>
> Second: from then on there are **two databases that do not talk**. Work done on one is invisible to the other and there is no merge. Decide which machine is authoritative *before* anyone starts working, not after.

Updating to a new version

A new version arrives as another zip, with another number (`ner-analyzer-4.8.1-windows.zip`). The `CHANGELOG.md` inside says what changed. Install it exactly like the first time: unzip, right-click `setup.bat` → *Run as administrator*, or double-click `setup-noadmin.bat` (with the same target folder, if you chose one).

1. Setup recognises the installation and says `Replacing version 3.x.y - its data is kept.` No administrator rights are needed: `setup-noadmin.bat` updates an installation made with or without them, for the account signed in, and keeps every project, account and drawing.
2. It holds both tasks, backs up the database into `data\backup-…\` , stops the server, moves the old installation aside to `ner-analyzer.previous` and moves its `data\` into the new folder.
3. It installs the new version in the now-empty folder, registers the tasks again, starts the server, opens the browser and deletes the package. The database migration, if the new version needs one, runs by itself on first start.
4. In the browser, `Ctrl` + `Shift` + `R` once: the sidebar shows the new number and the amber *the application was updated…* pill goes.
5. Once the new version works, delete `%LOCALAPPDATA%\Programs\ner-analyzer.previous` . The next update removes it anyway before keeping the one it replaces.

To go back to the previous version by hand: `stop.bat` , rename `ner-analyzer` to anything else, rename `ner-analyzer.previous` to `ner-analyzer` , and move the `data\` folder from the first into the second. If the database was already migrated by the new version, restore the backup setup took (`backup.bat --restore data\backup-…`) instead of using the migrated one.

Uninstalling

1. If you want to keep the data: `backup.bat --to` a folder outside the application.
2. In the installed folder, `autostart.bat remove` (removes both scheduled tasks) and `stop.bat` .
3. Delete the `ner-analyzer` folder (`%LOCALAPPDATA%\Programs\ner-analyzer` on an installation made by `setup.bat`), the `ner-analyzer.previous` folder beside it if there is one, and the **NER Analyzer** desktop shortcut. There is nothing else: no registry entry, no service, no file outside those folders. The application's Python lived inside it and goes with it.

Common problems

SYMPTOM	CAUSE AND FIX
`Forgot the password`	An administrator resets it in the app (*Account & security* → `Reset password`). With no administrator — or when the administrator is the one who forgot — run `reset-password.bat` in the installed folder (Mac: `mac/reset-password.command`): it lists the accounts, asks which one and the new password twice, enables the account if it was disabled and signs out its open sessions. It works with the server running.
`Could not unpack python-3.12.10-embed-amd64.zip`	The zip arrived incomplete or corrupted. Check it with `certutil -hashfile … SHA256` against the `.sha256` file and copy it again.
`The bundled Python could not start`	A security policy (AppLocker, application control) blocks programs in that folder. Try `C:\NER` ; if it persists, the machine's administrator has to allow the folder — `python.exe` is signed by the Python Software Foundation, which is usually enough for a publisher rule.
`DLL load failed while importing …`	The Visual C++ runtime (`msvcp140.dll`) is missing and the fallback copy could not be used. Install Microsoft's *Visual C++ Redistributable 2015–2022 (x64)* (`vc_redist.x64.exe` ; that one does need an administrator) and run `install.bat` again.
`The previous runtime is in use`	The server is running from `.python\` . `stop.bat` , then `install.bat` .
`Python 3.11 or newer was not found`	Only on a copy *without* `vendor\python\` (a development checkout). Use the installation package, which carries Python, or install 64-bit Python 3.12 on the development machine.
`Could not create the environment`	A development checkout in a OneDrive, Dropbox or network-drive folder. Move it to `C:\NER` .
`The browser says the site cannot be reached`	The server is not running, or is still starting (20 to 40 s). Wait and reload; if it persists, `start.bat` and read `data\server.log` .
`is already serving — nothing to do`	In the log: another server is already on the port. Not an error; the second copy exits quietly.

SYMPTOM	CAUSE AND FIX
`Port 8000 taken by another program`	`start.bat --port 8080`; for the tasks, `NER_PORT`.
`A blank page, or an amber pill in the top bar`	The browser cached an older interface: `Ctrl`+`Shift`+`R`. If the page stays white, press `F12` and look at the *Console* tab: the red message says what the browser refused. The server states the type of the interface files itself (`.js` as JavaScript), without relying on the Windows registry, which on many machines says `text/plain` and would make the browser refuse the interface silently.
`MANIFEST.txt is missing: this folder is not an installation package`	`setup.bat` was run from a copy of the code that is not a built package (a development checkout, or a hand-made zip). Use the package from `build_package.bat`, or `install.bat` in the folder itself.
`The target is inside the unzipped package · The unzipped package is inside the target folder`	The target folder and the unzipped folder cannot contain one another, because the unzipped one is deleted at the end. Unzip into Downloads and run again, or give another target folder.
`[*] Windows refused the sign-in task`	A policy of the machine does not let this account register even its own sign-in task; the line above the message is what Windows answered. The watchdog was installed all the same and starts the server within two minutes of every sign-in — nothing else to do. `schtasks /query /tn "NER Analyzer watchdog"` shows it.
`Could not create the watchdog task`	Windows refused the watchdog (the line above says why — most often a task of that name made earlier from an administrator window, which this account may not replace). Run `autostart.bat` once from a window opened with *Run as administrator*, or delete the old task in Task Scheduler. Until then, `start.bat` starts the application by hand.
`A drawing does not open after a restore`	The backup's `uploads` folder did not come with the database, or a file in it is not the one the project recorded. The server moves a record only to a file that is in this installation's `data\uploads` and is the same file; otherwise it leaves the record as it was and says so in `data\server.log` (`upload paths: …`). Restore the complete backup folder.
`The server did not answer within 90 s`	`setup.bat` installed and registered the tasks, but the server was not answering yet. Almost always a slow cold start: wait another minute and open `http://127.0.0.1:8000`; the watchdog keeps trying every two minutes. If it does not come, `data\server.log` in the installed folder says why.
`The unzipped folder is still there, empty`	A Command Prompt or an Explorer window was open inside it when `setup.bat` finished; Windows does not remove a folder in use. The files were deleted; delete the folder.
`Windows SmartScreen or *Open File — Security Warning* before a .bat`	The zip was downloaded, and Windows marks everything unzipped from it. Click *More info* → *Run anyway* (or *Run*). To avoid it, unblock the zip before unzipping: right-click → *Properties* → *Unblock*. The scripts are plain text; open them in Notepad to read what they do.
`The administrator rights were not granted`	The Windows prompt was answered *No*, closed, or the account cannot get them. Setup then asks *Install without administrator rights, for this account only? [Y/N]*: answer `Y` and it carries on in the same window. `N` installs nothing — run `setup.bat` as administrator, or `setup-noadmin.bat`, later.
`This window runs as another account than the person signed in`	Someone typed another account's administrator name and password into the Windows prompt. The window then runs as that account, and the application would be installed in its profile, where the person at the keyboard cannot use it. The line under the message shows both accounts. Grant administrator rights to the person who will use the application (temporarily is enough), sign in as that person, and run setup again.
`The old installation could not be moved`	On an update, something still has a file of the installed folder open: the server that did not stop, an Explorer window or a Command Prompt inside the folder, an editor. Close it and run setup again; nothing was changed. `… could not be removed` is the same for the `ner-analyzer.previous` folder an earlier update left: close what has it open, or delete it by hand.
`The page does not work in one browser`	Chrome, Edge and Firefox in a current version are supported. Update the browser, then `Ctrl`+`Shift`+`R`. Safari and Internet Explorer are not supported.
`The server vanishes with no message`	A process killed from outside: memory during a large export, the laptop leaving Modern Standby, or a policy of the machine itself. The watchdog starts it again within two minutes and records in `data\watchdog.log` the machine's memory and the standby and power events of the previous half hour — which is what says which of the three it was. Close the browser and Outlook before exporting a whole book.
`ModuleNotFoundError: pymupdf (or another package) after a successful install`	Another Python on the machine left packages under `%APPDATA%\Python\Python312\site-packages` (a `pip install --user` at some point), and the installer's pip took them as "already satisfied" while the server, which reads only `.python\`, never saw them. The installer runs pip without the user site and reinstalls every package into the runtime regardless, so this does not happen on a fresh install. On an installation that already shows it: `stop.bat`, then from the installed folder `.python\python.exe -s -m pip install --no-index --find-links vendor\wheels --no-deps --ignore-installed -r requirements-lock-win.txt` (with the package's `vendor\wheels` unzipped beside it), then `start.bat`. Deleting the stray `%APPDATA%\Python` folder is optional and harmless to this application.
`A black window flashes every two minutes`	The watchdog was registered by hand with `powershell.exe` instead of through `autostart.bat`. Run `autostart.bat` again: it registers the task through the `.vbs`, with no console.

Building the package

For whoever maintains the application: how to produce a version's zip from the development installation.

1. The version is in `VERSION`, in `frontend\package.json` and at the top of `CHANGELOG.md` — all three equal. `run-tests.bat` refuses a mismatch.
2. The compiled screens must carry that version: `cd frontend && npm run build` (only the development machine needs Node.js).
3. Double-click **`build_package.bat`**. It needs the internet *once*: it downloads into `vendor\wheels` the Python packages of `requirements-lock-win.txt` for Python 3.12 / 64-bit Windows (~100 MB, `pip` included) and calls `tools\make_package.py`, which fetches the python.org *embeddable package* into `vendor\python` (11 MB, once) and writes `dist\ner-analyzer-<version>-windows.zip` and the `.sha256`. `build_package.bat --keep` reuses the packages already downloaded.

The builder **refuses** to build when: the `data` folder or any database, log or upload would enter the package; a file carries a personal Windows path (`C:\Users\…`); the compiled screens or the manuals do not carry the version in `VERSION`; the Python zip does not match the SHA-256 pinned in `tools\make_package.py`; the `pip` or `numpy` wheel is missing from `vendor\wheels`; or a required file is missing. The unpacked `.python\` folder never enters the package — only the zip it is born from. The single-machine diagnostic scripts under `tools\` stay out by list; only the operating ones travel. `setup.bat` and `setup-noadmin.bat` are required in the package.

The macOS package is built by the same packager, on any system: `python tools/make_package.py --platform mac` → `dist/ner-analyzer-<version>-macos-arm64.zip`. It needs `vendor/python-mac/cpython-3.12.15+20261009-aarch64-apple-darwin-install_only.tar.gz` (fetched once from GitHub and refused unless it hashes to the pinned value) and `vendor/wheels-mac/`, filled with `pip download --platform macosx_11_0_arm64 --platform macosx_11_0_universal2 --python-version 3.12 --implementation cp --only-binary=:all: --no-deps -r requireme`. It refuses a Windows wheel, a pin without exactly one macOS wheel, and a `requirements-lock-mac.txt` that drifts from the Windows lock. Every `.command` is written with its executable bit, so Archive Utility restores it.

Two editions

Both are built from the same source by `tools\make_package.py`. The **team edition** (the default) carries the private reference material of the folder `backend\app\reference_private\`: a client's list of space names and naming table, when the builder has them. The **public edition** — `python tools\make_package.py --public`, written as `ner-analyzer-<version>-public-windows.zip` or `ner-analyzer-<version>-public-macos-arm64.zip` — leaves that folder out (only its `README.md` travels) along with the archived manual under `docs\history\`, and refuses to build while any shipped text names an event, an organiser, a venue or an employer. The application works the same without the private material: the TEAP names alone filter the name-only room labels, and the *Naming* panel shows the application's own naming rule.

Version and licence

This manual accompanies **version 4.8.1** of the NER Spatial Coverage Analyzer. Versions follow *MAJOR.MINOR.PATCH*: a correction that changes no screen and no file bumps the last number (2.0.1); a new feature that keeps everything valid bumps the middle one (2.1.0); a change an existing installation cannot absorb without intervention bumps the first (3.0.0). `CHANGELOG.md` records each one.

Created by Marcelo Barbosa. © 2026 Marcelo Barbosa. The application and its manuals are free software under the GNU Affero General Public License, version 3 (`LICENSE.txt`): anyone may use, study, copy, change and share them, and no written authorisation is needed. The author credit *Created by Marcelo Barbosa* is an additional term of the licence (section 7(b), in `NOTICE.txt`): every copy and every modified version keeps it. `THIRD-PARTY-NOTICES.txt` lists every component's licence. Third-party components (Python under `vendor\python`, under the Python Software Foundation licence; the Python packages under `vendor\wheels`; the JavaScript libraries compiled into the interface) remain under their own licences.